



Language-Agnostic Static Analysis of Probabilistic Programs

Markus Böck
markus.h.boeck@tuwien.ac.at
TU Wien
Vienna, Austria

Michael Schröder
michael.schroeder@tuwien.ac.at
TU Wien
Vienna, Austria

Jürgen Cito
juergen.cito@tuwien.ac.at
TU Wien
Vienna, Austria

ABSTRACT

Probabilistic programming allows developers to focus on the modeling aspect in the Bayesian workflow by abstracting away the posterior inference machinery. In practice, however, programming errors specific to the probabilistic environment are hard to fix without deep knowledge of the underlying systems. Like in classical software engineering, static program analysis methods could be employed to catch many of these errors. In this work, we present the first framework to formulate static analyses for probabilistic programs in a language-agnostic manner: LASAPP. While prior work focused on specific languages, all analyses written with our framework can be readily applied to new languages by adding easy-to-implement API bindings. Our prototype supports five popular probabilistic programming languages out-of-the-box. We demonstrate the effectiveness and expressiveness of the LASAPP framework by presenting four provably-correct language-agnostic probabilistic program analyses that address problems discussed in the literature and evaluate them on over 200 real-world programs.

CCS CONCEPTS

• **Mathematics of computing** → Bayesian computation; • **Theory of computation** → Program analysis; • **Software and its engineering** → Abstraction, modeling and modularity.

KEYWORDS

probabilistic programming, language-agnostic, program analysis

ACM Reference Format:

Markus Böck, Michael Schröder, and Jürgen Cito. 2024. Language-Agnostic Static Analysis of Probabilistic Programs. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*, October 27–November 1, 2024, Sacramento, CA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3691620.3695031>

1 INTRODUCTION

In Bayesian inference, we have a set of latent variables, observed data, and a statistical model which typically encodes how the observations are believed to be generated from the latents. Domain knowledge about the latents can be incorporated by specifying informative prior distributions. Posterior inference is concerned with finding or approximating the posterior probability distribution of the model—the distribution over the latent variables given

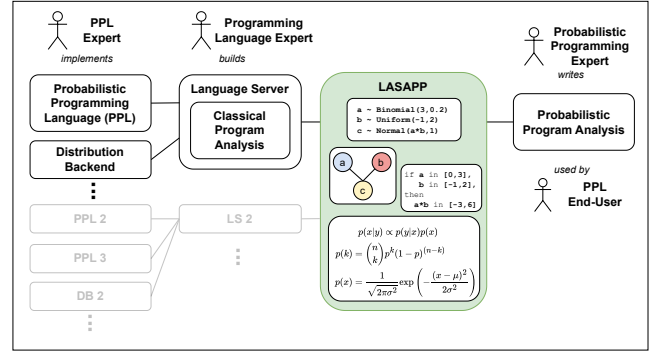


Figure 1: Overview of the LASAPP framework: Analyses are formulated in a high-level language-agnostic API. Internally, a language server performs classic analyses like data and control-flow analysis. Servers have to be built only once per host language and facilitate easy-to-implement bindings for multiple embedded probabilistic programming languages.

the observed data. Probabilistic programming systems provide an intuitive means to specify Bayesian models as simple programs and to automatically perform posterior inference on them. These systems decouple modeling from inference by implementing general-purpose inference algorithms. While previously the analysis of complex Bayesian models was reserved for experts, probabilistic programming enables it for users without substantial knowledge of statistics and inference techniques.

Writing probabilistic programs seems intuitive, but it also leads to many counter-intuitive situations. For instance, a programmer modeling a generative process may think about it sequentially, but the execution order of sample statements during inference need not be sequential and in general depends on the applied algorithm. Also, due to the stochastic nature of probabilistic programming, bugs may not occur during every run of the program and could be hard to reproduce. Furthermore, probabilistic programming systems abstract away thousands of lines of code and one may have to trace bugs deep into the inference libraries. For these reasons, debugging probabilistic programs still requires extensive knowledge of inference algorithms and the system at hand.

Program analysis can serve as a powerful technique to catch bugs statically and to improve the usability of probabilistic programming languages (PPLs) in general. In fact, recent research realizing this idea includes SlicStan, which relaxes the syntax of Stan [24], automatic guide generation for Pyro [39], improved debugging with probabilistic assertions [45], and slicing of probabilistic programs for improved inference [30]. However, a shared attribute of all of these methods is their limitation to a single probabilistic programming language.



This work is licensed under a Creative Commons Attribution-ShareAlike International 4.0 License.
ASE '24, October 27–November 1, 2024, Sacramento, CA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1248-7/24/10
<https://doi.org/10.1145/3691620.3695031>

In this paper, we present LASAPP (Figure 1)—the first approach to writing static analyses for probabilistic programs in a language-agnostic way. The key insight lies in the fact that while probabilistic programming languages differ in their implementation and expressiveness, the underlying meaning of these programs remains the same: a probabilistic model. By using existing program analysis techniques, like data and control-flow analysis, we can put an abstraction layer on top of classical program properties and formulate analyses of probabilistic properties in the language of these abstractions at a higher level. Our approach makes it easy to add support for new languages and thus existing analyses written in LASAPP are immediately available for them. Furthermore, the clear separation of classical program analyses entails that one can write probabilistic analysis without having to be a programming languages expert. We demonstrate that our framework allows the formulation of many practical and provably-sound analyses.

The main contributions of this work are:

- The first language-agnostic static analysis framework for probabilistic programs (LASAPP).
- Four program analyses, rooted in existing research, formulated in a language-agnostic way in LASAPP to demonstrate its capabilities:
 - Statistical Dependency Analysis (Section 4.2)
 - HMC Assumptions Checker (Section 4.3)
 - Parameter Constraint Verifier (Section 4.4)
 - Model-Guide Validation (Section 4.5)
- Correctness proofs of these analyses for a formal PPL,
- Prototype LASAPP bindings for the popular languages Pyro [8], PyMC [51], BeanMachine [55], Turing [21], and Gen [14], available at <https://github.com/lasapp/lasapp> and Zenodo [9].
- Evaluation of the practicability of the analyses on over 200 real-world probabilistic programs in four languages.

2 MOTIVATION

2.1 Probabilistic Programming

Probabilistic programming is an intuitive means to specify Bayesian models as programs. We start by giving a general definition:

DEFINITION. A probabilistic program is a (non-deterministic) program, for which an (unnormalized) weight can be assigned to each possible execution.

If the program returns a number, we can think of it as an (unnormalized) probability distribution over the return values. However, in probabilistic programming the local random variables of a program are also of great interest as they typically encode how observed data is believed to be generated. For each run of the program, the values of these variables are captured in an *execution trace*. Thus, probabilistic programs can also be viewed as an (unnormalized) joint probability distribution over all random variables declared in the program, or as an (unnormalized) probability distribution over execution traces [2, 23, 38, 54]. This distribution may also be referred to as *program density*.

In most probabilistic programming languages (PPLs), random variables of a model and their prior distributions are declared with special syntax in the form of *sample statements*. Relating these variables with program constructs implicitly defines the likelihood

of the model. Furthermore, in probabilistic programming systems, there is a mechanism to constrain the values of random variables to observed data. This results in a probabilistic model *conditioned* on data. All variables that are not constrained to any observed values are called *latent variables*.

```
def linear_model(x, y):
    a = pyro.sample("a", dist.Normal(0,10))
    b = pyro.sample("b", dist.Normal(0,10))
    s = pyro.sample("s", dist.InverseGamma(1,1))
    for i in range(len(x)):
        pyro.sample(f"y_{i}", dist.Normal(a*x[i]+b,s), obs=y[i])
```

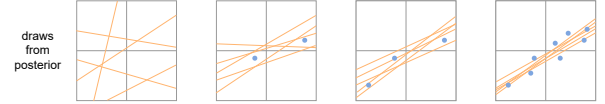


Figure 2: Linear model implemented in Pyro on the top. Results from posterior inference with decreasing uncertainty on the bottom.

In Figure 2, you can see a classic example of a probabilistic model, namely a linear model, where the uni-variate data y is believed to have a noisy linear relationship with the input values x . The variables y_i are constrained to observed data and we are interested in the posterior distribution over the latents a , b , and s —the distribution over the slope, intercept, and noise parameter given the data. We can inspect the posterior distribution to find the line that best fits the data and quantify the uncertainty about our conclusions.

What makes probabilistic programming special is that across different languages the semantic meaning of the programs remains the same. The languages differ in syntax and implementations, making trade-offs between the efficiency of inference algorithms and generality in terms of which models are expressible. However, the programs written in any language can all be interpreted as mathematical probabilistic models. Thus, the same probabilistic program analysis is meaningful in several languages and it is desirable to be able to apply the analysis directly without having to build up an analysis framework for each language.

2.2 Static Analysis of Probabilistic Programs

Consider a simple Bayesian model where a system can be in two states—5 or 6. Noisy measurements X of the state are available and with posterior inference we wish to find the most probable state of the system given the measurements and quantify the uncertainty. In Figure 3, you can see implementations of this model in Turing, BeanMachine, and Pyro.

Even though Hamiltonian Monte Carlo (HMC) [28] is a very robust and frequently recommended inference algorithm, it would be ill-advised to apply it for the described model. In fact, for all of the three implementations, it leads to cryptic runtime errors which need to be traced deep into the inference library. The trained eye will immediately spot the declarations of the discrete random variable state and find it incompatible with the gradient-based inference algorithm.

However, we argue that this is a non-trivial bug, because the error messages provide little help, it requires knowledge of the

```

@model function model(X)
  state ~ Categorical([0.5, 0.5])
  if state == 1      ⇒ ERROR: InexactError:
    mu = 5.          Int64{0.13392275765318448}
  else
    mu = 6.
  end
  for i in eachindex(X)
    X[i] ~ Normal(mu, 1.)
  end
end

@bm.random_variable
def state():
  return dist.Categorical(torch.tensor([0.5, 0.5]))
@bm.random_variable      ⇒ RuntimeError: only
def model(n):            Tensors of a floating
  if state() == 1:        point and complex dtype
    mu = 5.              can require gradients
  else:
    mu = 6.
  return dist.Normal(mu * torch.ones(n), 1.)

def model(X):
  state = pyro.sample("state", dist.Categorical([0.5, 0.5]))
  if state == 1:          ⇒ RuntimeError: Boolean
    mu = 5.              value of a Tensor with
  else:                  more than one value is
    mu = 6.              ambiguous.
  pyro.sample("X",
    dist.Normal(mu * torch.ones(len(X)), 1.), obs=X)

```

Figure 3: Simple model implemented in Turing v0.29.3, Bean-Machine v0.2.0, and Pyro v1.8.5 (top to bottom). Applying the HMC algorithm to these models results in different cryptic bugs. The root cause is identical for all of them: the declaration of the discrete variable `state` is not supported by HMC which requires continuous variables.

assumptions of the inference algorithm to fix, and the same source code would be correct for different inference algorithms.

Having observed the differences in syntax and disregarding the implementation details of HMC, we note that the underlying root cause of the runtime error is the same for all implementations. Even more, the bug can be caught *statically*. However, implementing such a static analysis separately for each PPL would require a separate parser for each language’s unique syntax, extracting and traversing different abstract syntax trees (ASTs) to find the respective sample statements, extracting the underlying distribution and looking up its properties based on the PPL’s particular semantics, and finally raising a warning if the type is discrete.

In this paper, we propose a framework where we abstract away much of the classical program analysis details like traversing the AST and parsing sample statements. As a result, the same static analysis can be written at a higher level in just three steps:

- (1) Find all random variable declarations.
- (2) Iterate over all random variables.
- (3) If a variable has a discrete distribution, raise a warning.

In terms of the concrete LASAPP API, these high-level steps translate almost one-to-one to Python code:

```

import lasapp
program = lasapp.ProbabilisticProgram(path)
random_vars = program.get_random_variables()      # (1)
for rv in random_vars:                            # (2)
  props = lasapp.infer_distribution_properties(rv)
  if props.is_discrete():                          # (3)
    raise Warning("...")

```

What makes our approach powerful is that adding support for new languages is easy and all existing program analyses written in the framework are immediately available. The above analysis is written in a completely language-agnostic way and can be applied to prevent all bugs from Figure 3. In fact, it also prevents two issues reported by developers in the Turing and Pyro forum, which were the inspiration for the discussed examples [20, 31].

3 LASAPP

Figure 1 presents an overview of LASAPP, our framework for Language-Agnostic Static Analysis of Probabilistic Programs. LASAPP targets PPLs that are embedded in a host language, like Pyro (which is embedded in Python), or Turing (embedded in Julia). Focusing on these types of PPLs allows us to separate probabilistic analysis from the classical program analysis machinery. LASAPP makes it possible for PPL experts to write high-level probabilistic program analyses in a language-agnostic way.

For each host language, a programming language expert implements a *language server*,¹ which is responsible for parsing source code and representing it as an abstract syntax tree, performing data- and control-flow analysis, abstract interpretation, or symbolic execution. Language servers need to be implemented only once per host language, and can be built using a plethora of well-established methods (Section 3.2). Once a host language is supported, adding support for new PPLs embedded in that language is simple. Mostly, one has to write bindings for a PPL’s special sample syntax (Section 3.1.1) and for its probabilistic distribution back-end (Section 3.1.2). For our prototype, we implemented language servers for Python and Julia, and added support for five PPLs, including two shared probability distribution back-ends (Table 1).

Table 1: Lines of code needed to add LASAPP support for various PPLs and probability distribution back-ends.

	Back-end / PPL	LOC	
Python	Language Server	1549	
	PyMC [51]	281	custom back-end
	torch.distributions [48]	67	shared back-end
	Pyro [8]	133	
	BeanMachine [55]	95	
Julia	Language Server	2175	
	Distributions.jl [7]	240	shared back-end
	Gen [14]	227	
	Turing [21]	172	

¹Not to be confused with the Language Server Protocol (LSP) which enables the use of static analysis tools independent of programming environment. One could use LSP to integrate LASAPP in an IDE. In contrast to LSP, LASAPP’s API facilitates the implementation of static analyses independent of probabilistic programming language.

3.1 Probabilistic Analysis API

LASAPP provides a Python API to implement static analyses of probabilistic programs. Figure 4 shows the main data types and most important API methods. We arrived at this API design by considering the common characteristics of popular PPLs to find a shared set of universal abstractions that would serve the needs of real-world probabilistic analyses (see Section 4).

3.1.1 Sample Statements. Different PPLs use different syntax to declare random variables. In these sample statements, we require a user-defined random variable name and an expression corresponding to the distribution of the variable. The variable name is also referred to as *address*—a (dynamically) unique string, with which the value of a variable is stored in the execution trace.

We give a few examples of different sample syntaxes, where we highlight the **addresses**, **distributions**, and **special syntax**:

- Turing has a special syntax that includes a tilde character, which is resolved with a macro before compilation. Values are assigned to program variables denoted on the left-hand side, which also serve as addresses:

```
mu ~ Normal(0., 1.)
x[i] ~ Normal(mu, 1.)
```

- In BeanMachine, a special decorator is added to function definitions to declare a random variable, which can then be referenced from anywhere with a regular function call. The above example can be rewritten in BeanMachine as:

```
@bm.random_variable
def mu():
    return dist.Normal(0., 1.)

@bm.random_variable
def x(i):
    return dist.Normal(mu(), 1.)
```

- In Pyro, sample statements are calls to the `pyro.sample` method, with addresses and distributions passed as arguments. In Pyro the example becomes:

```
mu = pyro.sample("mu", dist.Normal(0., 1.))
x[i] = pyro.sample(f"x_{i}", dist.Normal(mu, 1.))
```

LASAPP abstracts all of these into the same `RandomVariable` data type, preserving the semantics of the original syntaxes.

In this work, we only consider languages where sample statements can be extracted statically from the source code. To be able to perform classical static analyses, like data-flow analysis, we also require that random variables are always bound to program identifiers, e.g., variables or functions.

In addition to its distribution, it is also interesting to know whether a random variable is observed or not. Mechanism to declare an observed variable range from adding an optional “observed” parameter to the sample call, passing the observed value as a function parameter, or collecting the observations in a hash-map. For the sake of brevity we refer to the prototype implementation and PPL documentations for more details.

Lastly, probabilistic programs usually have a model entry-point or a program construct which encapsulates the sample statements.

For instance, in Pyro and Turing a program function defines a model while in PyMC sample statements are listed in the body of a `with` statement.

3.1.2 Distribution Back-ends. Probabilistic programming systems either have their own implementation of common probability distributions or use a package of their host language. For instance, PyMC has its own distribution library, while Pyro and BeanMachine share the PyTorch `torch.distributions` back-end [48] and Turing and Gen share the `Distributions.jl` back-end [7]. Sharing back-ends between PPLs allows for further modularisation and reusability but introduces an additional layer of variation.

LASAPP needs to map back-end-specific distribution and parameter names to a common vocabulary. For instance, consider the Gamma object in the `Distributions.jl` package, where the default parameters map to a parametrization in terms of shape and rate:

$$\text{Gamma}(\overset{\alpha}{1.5}, \overset{\beta}{2.0}) \rightsquigarrow p(x) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} \exp(-\beta x)$$

In PyTorch, however, the Gamma distribution is expressed in terms of shape and scale:

$$\text{Gamma}(\overset{k}{1.5}, \overset{\theta}{2.0}) \rightsquigarrow p(x) = \frac{1}{\Gamma(k)\theta^k} x^{k-1} \exp\left(-\frac{x}{\theta}\right)$$

LASAPP represents distributions with unique names for both parameters and the distribution itself.

3.2 Classical Analysis Foundation

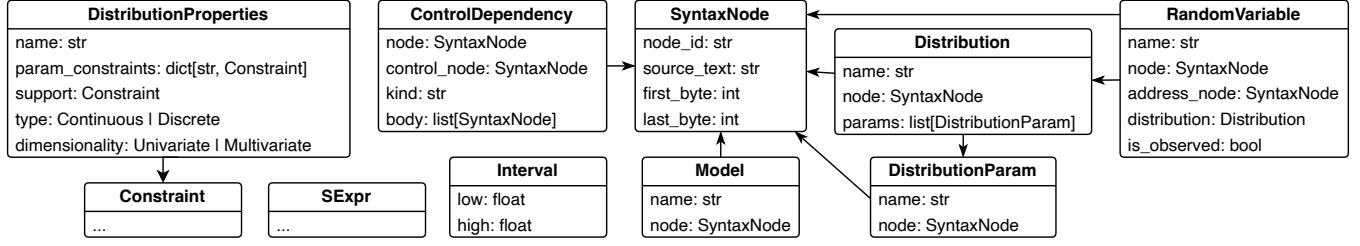
LASAPP’s probabilistic analysis API is built on a foundation of classical static analysis methods. This layered approach allows PPL experts to focus on implementing high-level language-agnostic analyses of probabilistic properties, while programming language experts can provide the necessary classical analysis foundations for each host language.

While all the classical techniques we discuss in this section are of general importance to PPL analysis, they were specifically chosen to aid in the implementation of the probabilistic analyses we present in Section 4. Other probabilistic analyses may require the support of additional classical techniques. By virtue of its modular design, LASAPP can be extended to incorporate additional static analysis methods in its language servers and expose them via the high-level API, accommodating the needs of new probabilistic analyses as they arise.

3.2.1 Data/Control-Flow Analysis. Data and control-flow analysis is concerned with finding the relationship between program variables and the order in which program statements are executed. We begin by informally defining data dependencies following [27].

DEFINITION (DATA DEPENDENCY). A statement S_1 is data dependent on statement S_2 , if S_2 produces a result that may be used by S_1 .

This definition is related to the notion of *reaching definitions* [11]. Note that if statement S_2 is data dependent on statement S_3 , then S_1 is also data dependent on S_3 by transitive closure.



```

class ProbabilisticProgram(filename: str)
    def get_model() → Model # see Section 3.1.1
    def get_guide() → Model # see Section 3.1.1
    def get_random_variables() → list[RandomVariable] # see Section 3.1.1
    def get_data_dependencies(node: SyntaxNode) → list[SyntaxNode] # see Section 3.2.1
    def get_control_dependencies(node: SyntaxNode) → list[ControlDependency] # see Section 3.2.1
    def estimate_value_range(expr: SyntaxNode,
                             masks: dict[SyntaxNode, Interval]) → Interval # see Section 3.2.2
    def get_path_condition(node: SyntaxNode, root: SyntaxNode,
                           mask: dict[SyntaxNode, SExpr]) → SExpr # see Section 3.2.3
    def infer_distribution_properties(rv: RandomVariable) → Optional[DistributionProperties] # see Section 3.1.2
  
```

Figure 4: Main types and methods of the LASAPP API.

Next, we give a general definition of a control-flow graph [11]. For a more formal definition, see for instance [15].

DEFINITION (CONTROL-FLOW GRAPH). A control-flow graph is a directed graph that connects basic blocks of a program (sequence of operations always executed together) and encodes all paths that might be traversed through a program during its execution.

Data dependencies and the control-flow are particularly important in the analysis of probabilistic programs, because they are used to find dependencies between sample statements. In fact, knowing the dependencies of random variables allows us to represent programs as *Bayesian networks* or more generally as *probabilistic graphical models*. Graphical models are well-understood mathematical objects [34], which allows us to leverage existing methods to reason about probabilistic programs. In Section 4.2, we will show how to statically extract the model graph of a probabilistic programs following the data and control-flow.

3.2.2 Abstract Interpretation. Abstract interpretation is a technique for approximating the behaviour of a program by replacing its concrete semantics with *abstract* semantics [12]. An abstract domain is usually based on (partially) ordered sets or lattices. As an illustrative example, consider an analysis to determine whether a numerical variable is positive or negative. We can replace the concrete variables with an abstract representation of just their signs, i.e., whether the variable is positive (+), negative (−), or zero (0). To achieve soundness, the concrete semantics are over-approximated: we may need to accept that we cannot always tell the sign of a variable and have to assign it an abstract “don’t know” value (⊤).

For our purposes, we are interested in more than just the signs of program variables. As probabilistic programs are stochastic by nature and the values of random variables may be different from

one execution to another, we would rather like to know the possible *range of values* of any particular variable. To achieve this, we have implemented an interval abstract domain in LASAPP to approximate value ranges with intervals. Intervals are propagated through the program by following interval arithmetic [1, 44]. See some of the rules below.

$$[a, b] + [c, d] = [a + c, b + d]$$

$$[a, b] \times [c, d] = [\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)]$$

$$\exp([a, b]) = [\exp(a), \exp(b)]$$

In Section 4.4, we analyse the value range of distribution parameters to verify that they do not violate the imposed constraints in any possible program execution.

3.2.3 Symbolic Execution. Symbolic execution is similar to abstract interpretation, in the sense that here too the concrete semantics of a program are abstracted in order to prove certain properties. Program variables or inputs are exchanged with symbols representing arbitrary values. The syntax of the programming language remains the same, but the execution now follows the symbolic semantics [33]. In contrast to abstract interpretation, symbolic execution does not necessarily explore all possible program paths, but it has the advantage that it can often avoid over-approximation.

A particular application of symbolic execution relevant for this work is concerned with so-called *path conditions*. Depending on the values of variables appearing in conditional if-statements, either the “then” or the “else” branch will be taken during execution. By representing the conditional symbolically, we can derive constraints that need to be satisfied for a certain program branch to be executed. This is especially interesting for probabilistic programs that exhibit stochastic branching. For these types of programs, random variables may appear in conditional statements where a certain fraction p

of executions are expected to take the “then” branch, while the “else” branch will be taken with probability $1 - p$. Such properties are especially relevant for validating so-called guide programs, a practical analysis we describe in Section 4.5.

4 SOUND STATIC ANALYSES IN LASAPP

The goal of this section is to first, demonstrate that the LASAPP framework enables us to formulate non-trivial program analyses in a high-level way, and second, to show that these analyses can be proven to be sound. To this end, we present four static analyses rooted in existing research (see related work, Section 6.2) implemented in the LASAPP’s API. Furthermore, we introduce a formal PPL and establish correctness theorems for each of the analyses. We provide proofs in the supplementary material [9]. The practical usefulness of these analyses is then evaluated on real-world programs in Section 5. For the sake of brevity, we will only give an overview description of the analyses and simplified versions of the implementations in LASAPP’s API. The full implementations can be found in the replication package [9].

4.1 A Small Formal PPL: SIMPLEPROB

In this work, we not only consider the application of LASAPP to practical PPLs, but we also aim to investigate the framework from a formal stance. To this end, we introduce a small formal PPL called SIMPLEPROB, which is similar to other formal PPLs studied in prior work [24, 30]. Its complete syntax is given below.

Syntax of SIMPLEPROB:

$E ::=$	expression
c	constant
x	variable
$g(E_1, \dots, E_n)$	function call
$S ::=$	statement
skip	skip
$x = E$	assignment
$S_1; S_2$	sequence
if E then S_1 else S_2	if statement
$x = \text{sample}(E_0, f(E_1, \dots, E_n))$	sample
$\text{observe}(E_0, f(E_1, \dots, E_n))$	observe
$P ::=$	program
model S	model definition
model S_1 ; guide S_2	model and guide

In this section, we only give a brief overview of the semantics of SIMPLEPROB and the formalisation of the LASAPP API (Figure 4). More details are provided in the supplementary material, in particular, the full formalisation of the API can be found in Section 2.

The meaning of a SIMPLEPROB program P is a mapping from traces $\text{tr} \in \mathcal{T}$ to their density. A trace is a mapping from addresses to values $\text{tr}: \text{Strings} \rightarrow \mathcal{V}$. The density is computed according to small-step operational semantics defined for the control-flow graph (CFG) of P . The semantics are standard for the non-probabilistic language constructs. For a CFG node N corresponding to a sample statement $x = \text{sample}(E_0, f(E_1, \dots, E_n))$, the small-step relation is defined as

$$(\sigma, N) \xrightarrow{\text{tr}} (\sigma[x \mapsto V, \mathbf{p} \mapsto \sigma(\mathbf{p}) \times \text{pdf}_f(V; V_1, \dots, V_n)], N'),$$

where N' is the successor node, $V_i = \sigma(E_i)$ is the value of expression E_i according to the current state σ and $V = \text{tr}(V_0)$ is the value stored in the trace tr at address V_0 . In the updated state, the variable x gets assigned the trace value V and the current density $\mathbf{p}$ is multiplied with $\text{pdf}_f(V; V_1, \dots, V_n)$ — the density of distribution f evaluated with arguments $V_1, \dots, V_n$ at value V .

The formalisation is best illustrated by considering a simplified version of the program in Figure 3, rewritten in SIMPLEPROB:

```
model
  s = sample("s", Bernoulli(0.5))
  if s == 1 then
    m = 5.
  else
    m = 6.
  observe("x", Normal(m, 1.))
```

In Figure 5, you can see the control-flow graph of the program above with annotations for the program state at several nodes.

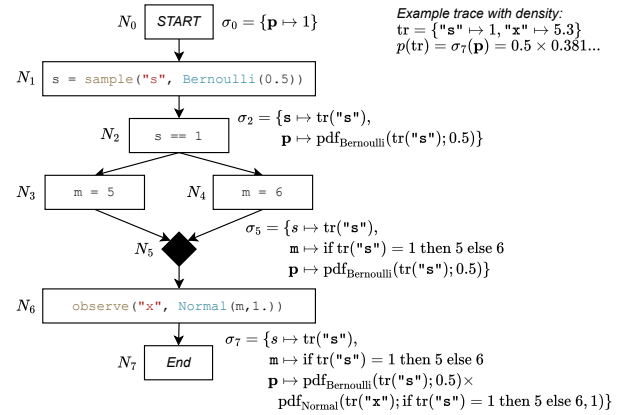


Figure 5: Control-flow graph of SIMPLEPROB program.

The control-flow graph representation is convenient for formally defining the LASAPP API (Figure 4) for SIMPLEPROB. To find the data dependencies of a variable x at CFG node N , we traverse all predecessor nodes until we encounter a node that assigns x . To find the data dependencies of an expression E we collect the dependencies for each variable read in E . For instance, $\text{Normal}(m, 1.)$ in N_6 is data dependent on N_3 and N_4 . Similarly, for control dependencies we traverse all predecessors and collect branch nodes which were not closed yet. For instance, N_3 and N_4 are control dependent on N_2 , but N_6 is not. Without going into too much detail, estimating value ranges or computing path conditions is done by replacing the trace values $\text{tr}(\dots)$ in the states σ with intervals or symbols, respectively.

4.2 Statistical Dependency Analysis

Finding dependencies between sample statements is a building block of many analyses [6, 10, 30]. In fact, representing a probabilistic program graphically as a Bayesian network allows us to apply algorithms designed specifically for probabilistic graphical models for further analysis [34]. The high-level overview of the algorithm to find these dependencies is described in five steps.

- (1) Initialise an empty statistical dependency graph $\mathcal{G}$.
- (2) Find all sample statements.
- (3) Filter for random variables that are reachable from the model entry point (e.g. by inspecting the call graph).
- (4) For each random variable, traverse the data dependency graph until we encounter another sample statement. For each data dependency d , we also need to traverse the data dependencies of every control dependency of d .
- (5) Add an edge between the two sample statements to $\mathcal{G}$. As the address of a random variable may be dynamic, we also need to get its dependencies.

You can see a concrete LASAPP implementation of the steps above in Listing 1 (simplified and shortened for clarity).

```
dependencies = [] # == G # (1)
all_variables = program.get_random_variables() # (2)
model = program.get_model()
random_vars = filter(all_variables, model) # (3)
for rv in random_vars: # (4)
    queue = [rv.address_node, rv.distribution_node]
    while len(queue) > 0:
        node = queue.popleft()
        data_deps = program.get_data_dependencies(node)
        for dep in data_deps: # (4)
            if dep in random_vars:
                dependencies.append((dep, rv)) # (5)
                queue.append(dep.address_node)
            else:
                queue.append(dep)
        for dep in program.get_control_dependencies(node): # (4)
            queue.append(dep.control_node)
```

Listing 1: Model graph extraction written in LASAPP.

In Figure 6, we illustrate how the algorithm operates and why we also need to traverse the data dependencies of control nodes. To find the statistical dependencies of the random variable x , we start by finding the data dependencies of the distribution parameter m . We notice that the value of m depends on the conditional $s == 1$ and thus, also x implicitly depends on the conditional. We continue to find the only data dependency of $s == 1$ in form of the sample statement of s , stop the traversal and add the edge $s \rightarrow x$ to the statistical dependency graph. In Figure 7, you can see an example program adapted from Mak et al. [41] and its static model graph.

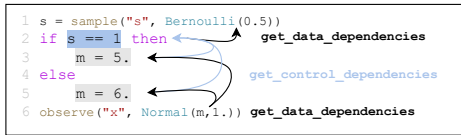


Figure 6: Repeated application of `get_data_dependencies` and `get_control_dependencies` to find statistical dependencies in a SIMPLEPROB program.

The analysis described above is sound for programs written in SIMPLEPROB in the sense of the following proposition.

PROPOSITION 4.1. *Let P be a SIMPLEPROB program. Then, the model density of P factorises according to the graph extracted with the analysis outlined in Listing 1.*

This statement will be made more precise in Section 3 of the supplementary material, where we also provide the proof.

```
@model function pedestrian(end_distance)
    start ~ Uniform(0,3)
    t = 0; distance = 0.0;
    position = start
    step = Dict()
    while position > 0 && position < 10
        t = t + 1
        step[t] ~ Uniform(-1, 1)
        distance = distance + abs(step[t])
        position = position + step[t]
    end
    end_distance ~ Normal(distance, 0.1)
    return start
end
```

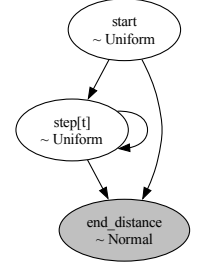


Figure 7: Static model graph of a Turing program extracted with LASAPP.

4.3 HMC Assumptions Checker

We turn back to our motivating example, see Figure 3. Recall that the runtime errors caused by applying the Hamiltonian Monte Carlo algorithm can be prevented statically by disallowing discrete variables in the model. We already presented the corresponding LASAPP analysis in Section 2 and include it again in Listing 2. However, the assumption of continuous variables is not the only one for HMC and we continue to implement more checks.

As a gradient-based method, HMC requires the program density to be differentiable with respect to all random variables. Stochastic control-flow often leads to discontinuous densities and can be statically avoided, if we check whether a random variable is control dependent on another variable. This check, among others, is implemented in Stan’s pedantic compiler mode [5] and we can adopt it to LASAPP with a slight modification of the dependency analysis of Section 4.2, such that we only record those dependencies that affect the control-flow. We abbreviate this subroutine with `has_random_control_deps`. The full implementation can be found in the supplied artifact and the supplementary material.

```
random_variables = program.get_random_variables()
for rv in random_variables:
    props = lasapp.infer_distribution_properties(rv)
    if props.is_discrete():
        raise Warning("Discrete Variables ...")
    if has_random_control_deps(rv):
        raise Warning("Stochastic Control-Flow ...")
```

Listing 2: HMC assumption checker written in LASAPP.

Again, we can make a formal statement about the soundness of the HMC assumption checker for SIMPLEPROB programs. See Section 4 of the supplementary material for the proof.

PROPOSITION 4.2. *Let P be a SIMPLEPROB program that only contains differentiable primitive functions and static addresses for sample statements. If the analysis outlined in this section does not produce warnings, then the model density defined by P is differentiable.*

4.4 Parameter Constraint Verifier

One of the bigger challenges when starting with probabilistic programming is that probabilistic programs are typically not executed sequentially and values of variables are random. This may lead to

runtime errors that occur with low probability which makes them hard to reproduce and debug. Therefore, it is often productive to think about the entire *range of values* a random variable can take (its support) to verify that a program runs without errors for every possible execution trace.

This line of thought motivated the use of abstract interpretation in form of interval analysis as described in Section 3.2.2 to verify that the parameters passed to distributions satisfy constraints. For example, the standard deviation parameter of a Normal distribution has to be strictly positive. Note that this check is also included in Stan’s pedantic mode implemented directly in the compiler [5]. In contrast, the analysis presented in the following is written in the LASAPP framework and can be applied to multiple PPLs at once.

We give a brief overview of the probabilistic analysis to find constraint violations in distribution arguments:

- (1) Initialise an empty “mask” $\mathcal{A}$.
- (2) Find all sample statements.
- (3) For each random variable R , retrieve its value range V and add $R \mapsto V$ to $\mathcal{A}$.
- (4) For each random variable, retrieve its parameter constraints.
- (5) Check for violations by comparing against the estimated range for the parameter expression.

The LASAPP implementation can be seen in Listing 3.

```
mask = {} # (1)
random_variables = program.get_random_variables() # (2)
for rv in random_variables:
    props = lasapp.infer_distribution_properties(rv)
    mask[rv.node] = Interval(props.support) # (3)

for rv in random_variables:
    props = lasapp.infer_distribution_properties(rv)
    for param in rv.distribution.params:
        constraint = props.param_constraints[param] # (4)
        estimated_range = program.estimate_value_range( # (5)
            expr=param.node,
            mask=mask
        )
        if not estimated_range.is_subset_of(constraint): # (5)
            raise Warning("Constraint violation ...")
```

Listing 3: Parameter constraint verifier written in LASAPP.

The analysis is best explained by considering the example program in Figure 8. We estimate the value range of the variable `prob`. The variable `z` is masked with $[-\infty, \infty]$ and `u` is masked with $[0, 1]$. Then, these intervals are transformed according to the mathematical expressions by applying interval arithmetic rules. Lastly, since either of the two branches could be taken, we compute the union of the estimated ranges for `prob` to be $[0, 1.5]$. This violates the constraint for the “success probability” of the Geometric distribution, which has to be less than 1. Note that the program would result in a runtime error with a probability of less than 0.1%.

In the following proposition, we state the soundness of the constraint verifier for SIMPLEPROB programs, which is proven in Section 5 of the supplementary material.

PROPOSITION 4.3. *Let P be a SIMPLEPROB program that only uses uni-variate random variables. If the analysis outlined in this section does not produce warnings, then P is free of distribution parameter constraint violations.*

```
@gen function model()
  b ~ bernoulli(0.999) # [0,1]
  if b
    z ~ normal(0.,1.) # Real
    prob = 1/(1+exp(z)) # [0,1]
  else
    u ~ beta(1,1) # [0,1]
    prob = 1.5 * u # [0,1.5]
  end
  x ~ geometric(prob) # [0,1.5]
end
WARNING: Parameter p of Geometric
distribution has constraint [0, 1],
but values are estimated to be in [0.0, 1.5].
```

Figure 8: Gen program with parameter constraint violations.

4.5 Model-Guide Validator

Many posterior inference algorithms targeting the density p make use of a reference distribution q . For instance, MCMC algorithms benefit from making random value draws not from the distribution defined in the model, but from a proposal distributions q . Furthermore, variational inference methods fit a so-called variational distribution q to be close to p . In these algorithms, the density ratios $p(x)/q(x)$ and $q(x)/p(x)$ occur, which are only well-defined (in a probabilistic sense) if p and q satisfy the absolute continuity property. We say density p is *absolutely continuous* with respect to density q and write $p \ll q$ if

$$p(x) > 0 \implies q(x) > 0 \text{ for all traces } x.$$

This condition makes $p(x)/q(x)$ well-defined, whereas $q \ll p$ makes $q(x)/p(x)$ well-defined (we take $0/0 = 0$).

In probabilistic programming, these reference distributions come in form of so-called *guide programs* [13, 39], which are typically defined by the user. To make posterior inference sound the guide must match the model in terms of absolute continuity. This motivates the formulation of a static analysis which automatically verifies that a model and guide program satisfy the absolute continuity property. Such an analysis was already developed for Pyro [37] and we formulate it in a language-agnostic manner in LASAPP.

This analysis is described briefly in five steps, where we assume that all random variables are uni-variate and that all addresses are static.

- (1) Find all sample statements for the model and guide program. To check $\text{model} \ll \text{guide}$ take $p = \text{model}$ and $q = \text{guide}$. To check $\text{guide} \ll \text{model}$ take $p = \text{guide}$ and $q = \text{model}$.
- (2) Get the path condition $\text{pc}(\text{stmt})$ for each sample statement where we mask random variables symbolically.
- (3) Express the support interval $[a, b]$ of sample statements with address X symbolically in form of distribution constraints $\text{dc}(\text{stmt}) = a \leq X \wedge X \leq b$.
- (4) Combine the constraints of all sample statements in a program p (model or guide) like below:

$$\text{constraints}(p) = \bigwedge_{\substack{\text{stmt} \in \text{sample-} \\ \text{stmts in } p}} (\text{pc}(\text{stmt}) \implies \text{dc}(\text{stmt}))$$

- (5) Use an SMT-solver (e.g. Z3 [16]) to prove that the following predicate is unsatisfiable:

$$\neg[\text{constraints}(p) \implies \text{constraints}(q)]$$

Again, we show a simplified version of the LASAPP implementation in Listing 4.

```
def get_dist_constraint(rv):
    props = lasapp.infer_distribution_properties(rv)
    a, b = Interval(props.support)
    X = SExpr(rv)
    return And(a < X, X < b)

rvs = program.get_random_variables() # (1)
P = program.get_model(); Q = program.get_guide();
# Q = program.get_model(); P = program.get_guide();
P_rvs = filter(rvs, P); Q_rvs = filter(rvs, Q);

A = {rv.node: SExpr(rv) for rv in rvs} # symbolic masks
pc = { # (2)
    **{rv: program.get_path_condition(rv.node, P.node, A)
        for rv in P_rvs},
    **{rv: program.get_path_condition(rv.node, Q.node, A)
        for rv in Q_rvs}
}

dc = {rv: get_dist_constraint(rv) for rv in rvs} # (3)

impl = Implies( # (4)
    And([Implies(pc[rv], dc[rv]) for rv in P_rvs]),
    And([Implies(pc[rv], dc[rv]) for rv in Q_rvs]),
)
if check(Not(impl)) == satisfiable: # (5)
    raise Warning("Absolute continuity violation ...")
```

Listing 4: Model-guide validation written in LASAPP.

The symbolic formula $\text{constraints}(p)$ precisely captures the condition $p(x) > 0$. This makes the analysis sound, which is proven for SIMPLEPROB programs in Section 6 of the supplementary material and is summarised in following proposition.

PROPOSITION 4.4. *Let P be a SIMPLEPROB program that only uses uni-variate random variables with static addresses. Let p be the model density and q the guide density defined by P . If the analysis outlined in this section does not produce warnings, then $p \ll q$, i.e., for all traces x it holds that $p(x) > 0 \implies q(x) > 0$.*

Note that it is often useful to further check whether sampling a random variable with address X in the model program implies sampling the same variable in the guide program. We have extended Listing 4 in the prototype implementation with this check and prove it to be correct in the supplementary material.

5 EVALUATION

While we have established correctness results of the presented static analyses for the small formal PPL SIMPLEPROB, it is natural to ask whether these analyses have also a practical benefit for the more complex real-world PPLs. To answer this question, we test the prototype implementation of the analyses on a plethora of real-world probabilistic programs. In Table 2, you can see a summary of the evaluation results described in this section.

To evaluate the statistical dependency analysis (section 4.2) and constraint verifier (section 4.4), we gathered a total of 117 Turing and 97 PyMC programs from open-source repositories [49, 53, 56]

Table 2: Summary table of evaluation results.

Analysis / PPL / data source	total	result		
Dependency Analysis				
Turing [53, 56]	117	111 correct	6 unsupp.	
PyMC [49]	97	95 correct	2 unsupp.	
Constraint Verifier				
Turing [53, 56]	117	80 verified	37 unsupp.	
PyMC [49]	97	65 verified	32 unsupp.	
HMC Assumption Checker				
Gen [5, 41, 43, 46, 60]	8	7 true pos.	1 false neg.	
Model-Guide Validator				
Pyro [37]	8	6 true neg.	2 true pos.	

ranging from a collection of tutorial programs to implementations of advanced statistical models and Bayesian data analyses, as well as case studies on Bayesian cognitive modeling [22, 32, 36, 42]. These repositories were listed on official Turing and PyMC homepages.

By manual inspection, we verified that the statistical dependency analysis correctly extracted the model graph of 111 Turing models. This was accomplished by deriving the mathematical expressions of the program densities by hand, according to Turing’s semantics, from which one can read off the dependencies. It failed on six programs, where programming constructs (generators and lambda functions) were used which are not supported by our prototype yet. For the PyMC programs, we compared the static model graph against the dynamically computed graph by PyMC. There were only errors for two models, due to arbitrary distribution expressions used in sample statements. While the error rate is already low, it could be further lowered by improving our prototype implementation, because the errors are not inherent limitations of the LASAPP framework itself. We also note that with this analysis we even found a bug in the implementation of one model.² In this program a random variable node was disconnected from all other variables in the graph, because a different variable was mistakenly used in its place in the computation of an intermediate value.

With the constraint verification analysis, we were able to assert that 80/117 Turing programs and 65/97 PyMC are free of parameter constraint violations. For the rest of programs, this assertion could not be established, because either 1) data read from an external source was required to satisfy constraints or 2) multivariate distributions were present where the constraints cannot be verified with interval arithmetic. The second reason could be mitigated by extending LASAPPs classical analysis foundation with a new abstract interpretation domain suited for the constraints of multivariate distributions. This exemplifies the benefits of the modular design of LASAPP.

To evaluate the HMC assumption checker (section 4.3), we cannot use any arbitrary probabilistic program, since not every program is meant to work with HMC. Instead, we collected eight probabilistic programs with discontinuous densities from literature. For six of them the discontinuity arises from random control-flow, for one of them from the use of discrete variables, and for one of

²https://github.com/lasapp/lasapp/blob/main/evaluation/turing/statistical_rethinking_2/chapter_14_6.jl

them from the use of the discontinuous floor rounding function. Seven of these programs served as motivating examples for more sophisticated inference methods that overcome the shortcomings of HMC [41, 43, 46, 60] and one program motivated the static analysis of random control-flow in the Stan compiler [5]. We implemented these models in Gen and found that our static analysis gives the correct warning that HMC is not applicable for 7/8 programs. It fails to give the warning for the program which uses the floor function as our analysis does not verify the continuity properties of primitive functions used in the program.

We evaluate the model-guide validator (section 4.5), on Pyro programs, as Pyro heavily relies on model-guide pairs to perform stochastic variational inference (SVI). In fact, Lee et al [37] developed a static analysis with the same goal and provide a benchmark set of eight Pyro programs, which we use for this evaluation. Even though their method is formally sound, more sophisticated, and specifically tailored to Pyro, our more light-weight language-agnostic analysis achieves the same result on this benchmark set. Namely, it asserts the validity of six model-guide pairs and raises the correct warnings for the `br` and `lda` programs.³

We end this section by noting that while the benchmark sizes for the HMC assumption checker and model-guide validator may seem small in comparison to the other benchmarks, they are typical for analyses that are only meaningful for certain probabilistic programs (i.e. those that intend to use HMC or guide programs). In contrast, the statistical dependency analysis and parameter constraint verifier are meaningful for any arbitrary probabilistic program. Lastly, we like to point out that running all analyses mentioned in this section takes a combined time of less than 15 seconds on a 2022 3.5 GHz ARM-based CPU with 32GB RAM.

6 RELATED WORK

6.1 Language-Agnostic Program Analysis

Research on static program analysis is plentiful [17, 26]. Therefore, in this section, we focus on program analysis methods that aim to support language-agnostic analyses to some extent.

In 2015, Google presented the Tricorder framework [50]. Similar to the LASAPP language servers, the Tricorder ecosystem consists of analyser worker services written in different languages that all implement a common language-agnostic API. Analysers may be written in any language and may analyse any language. The analysis driver calls out to the language- or compiler-specific workers to run the program analyses.

The Cobra tool by Hoffmann [29] achieves language-agnosticism by interpreting programs simply as a list of tokens. These tokens are annotated with categories (identifier, keyword, type, etc.) and with additional context information. A special query language was developed to formulate program analyses on these lists of tokens.

The domain-specific language and infrastructure Boa [18, 19] was developed for easier and reproducible analyses of software repositories. Boa implements a representation of source code as abstract syntax trees with generic nodes like declaration, type, method, etc. It provides language features inspired by visitor patterns to allow users to query and analyse the ASTs of large-scale projects.

³In the `lda` program, the model-guide mismatch is intentional, because variational expectation-maximisation was employed instead of SVI.

The methods listed above focus on program analyses for general program properties that are applicable to many languages and are not powerful enough to implement the analyses presented in this work. In contrast, our framework is designed specifically for the analysis of probabilistic programs written in different host-languages.

Tangentially related, there are efforts to create language-agnostic representations of source code for machine learning models. For instance, Zügner et al. [61] presented a multi-lingual code summarization model.

Lastly, we mention the compilation framework LLVM [35]. LLVM is designed around a language-independent intermediate representation to facilitate program analyses. However, this representation is too low-level for the analyses considered in this work.

6.2 Static Analysis For Probabilistic Programs

The field of static analysis for probabilistic programming is young and largely under-explored. In this section, we will highlight prior work related to the probabilistic program analyses presented in Section 4 and refer to Bernstein’s survey [4] for more methods and details.

In 2014, Hur et al. [30] presented a way to reduce probabilistic programs to only those statements that are relevant to estimate the distribution of the return expression based on the dependency structure of the program. Furthermore, it is well known that the graphical structure of a model can speed up inference [34, 47, 57] and help in program understanding [25]. These reasons motivated the dependency analysis of Section 4.2.

Also in 2014, Sampson et al. [52] introduced probabilistic assertion statements and an evaluation approach to either statically or dynamically verify them.

Stan has received the most attention with regards to static analysis. In 2018, SlicStan [24], a compositional version of the probabilistic programming language, was published. By relaxing the strict block syntax of Stan, it makes the code more flexible, reusable, and beginner friendly. Also, a semantics-preserving translation to Stan was developed. In the following years, Stan was also extended with a pedantic compilation mode [5]. Among other checks, warnings will be raised if there are any distribution usage issues or stochastic control-flow, which partly inspired the analyses in Sections 4.3 and 4.4. A static transformation method relying on the extraction of the factor graph was developed to make draws from the prior and posterior predictive distribution of Stan programs possible [6]. Lastly, a compilation scheme was designed to translate a Stan program to a generative PPL [3].

It is worth mentioning that the challenges for HMC caused by discontinuities in the program density is an active research field in itself [40, 41, 43, 46, 59, 60], which also served as a motivation for the analysis in Section 4.3.

Most recently Wang et al. [58] developed a type system to enforce absolute continuity of Pyro guide programs and Li et al. [39] presented an automatic approach to generate sound guide programs for a given Pyro program. Related to these methods, Lee et al. [37] developed a static analysis to verify that a guide matches the model for variational inference in Pyro. These papers inspired the model-guide validation analysis in Section 4.5.

While the analyses of Section 4 have some overlap with the methods above, the key difference is that through the LASAPP framework our analyses are not restricted to one particular probabilistic programming language, but are written in a language-agnostic way and can be applied to any PPL which implements the API bindings.

7 LIMITATIONS AND THREATS TO VALIDITY

One factor that could potentially limit the generalisability of our approach is that the framework and its abstractions were only evaluated on the selected set of probabilistic programming languages and the presented program analyses. However, we argue that LASAPP generalises to new PPLs and analyses. The fact that the framework supports many different kinds of sample statements (function calls, function definitions, special syntax) shows its flexibility and adaptability to new languages. Also, the classic static analyses that are the backbone of the API are versatile. In particular, data and control-flow was important for almost every program analysis presented in Section 4. Furthermore, the modularity of the LASAPP framework allows us to extend it with additional classic static analysis methods to support new PPLs or new analyses if needed.

As with any static analysis approach, these methods—and as a result LASAPP—are subject to the usual limitations. That is, we rely on the assumption that we can extract all the required information for a probabilistic program from its source code. This assumption is true for SIMPLEPROB by construction, but may not hold for real-world PPLs. As a contrived example, it is possible in some PPLs (e.g. PyMC) to write a program where the model structure depends on I/O. However, we found that the vast majority of probabilistic programs are written in a conventional way such that the assumption is fulfilled to the degree where the considered analyses can be applied.

Furthermore, while the analyses are proven to be sound for SIMPLEPROB, this cannot be guaranteed for the real-world PPLs, because they are more complex and there may be subtle differences in semantics. However, we evaluated the analyses to be practical and accurate for real-world programs and we have not identified any significant differences in semantics. But note that the selected benchmark programs for the evaluation may not be representative of all real-world probabilistic programming, even though we argue that they cover a wide range of applied Bayesian modeling.

Lastly, we like to point out that LASAPP is not limited to PPLs embedded in a host language. It is entirely possible to implement the language server and API for a standalone PPL to make the existing analyses available. However, because implementing the language server takes the most effort, LASAPP shines when it is possible to share the analysis back-end of a host language between multiple embedded PPLs.

8 CONCLUSION

In this work, we presented the first language-agnostic static analysis framework for probabilistic programs LASAPP. We have demonstrated the effectiveness and expressiveness of the LASAPP framework by providing prototype implementations for five state-of-the-art probabilistic programming languages (Pyro, PyMC, BeanMachine, Turing, and Gen) and by presenting four static program analyses that address problems discussed in literature. These analyses

include checking the assumptions of the HMC inference algorithm, verifying parameter constraints of distributions, and validating the compatibility of guide programs. We have proven the analyses to be sound for the formal PPL SIMPLEPROB and evaluated them on over 200 real-world probabilistic programs, implemented in four different languages, demonstrating their practical benefits.

Lastly, we summarise the key characteristics of our language-agnostic static analysis framework for probabilistic programming by reflecting on the insights gained from designing LASAPP. First, probabilistic programming languages are special in the sense that their programs can be universally interpreted as probabilistic models. They only differ in the underlying posterior inference machinery and expressiveness. This justifies the goal of a language-agnostic framework as the same analysis for some probabilistic program properties is meaningful for many PPLs.

It is clear that probabilistic program analyses rely on classical static analysis methods. By designing LASAPP in a modular fashion, we made an effort to clearly separate the two types of analysis. This allows programming languages experts to build the language servers and implement the needed classical static analysis techniques like data and control-flow analysis. The probabilistic programming systems experts can then provide the bindings for their PPL, where we have identified sample statements as the core language construct. The complexity of these implementations is then abstracted away by LASAPP’s API. We argue that the selection of LASAPP’s API methods forms a set of essential abstractions. Finding dependencies of random variables via data and control-flow is important for almost all analyses. Further, interval arithmetic and symbolic execution lends itself to handle the stochastic nature of probabilistic programs.

ACKNOWLEDGMENTS

The authors gratefully acknowledge the financial support under the scope of the COMET program within the K2 Center “Integrated Computational Material, Process and Product Engineering (IC-MPPE)” (Project No 886385). This program is supported by the Austrian Federal Ministries for Climate Action, Environment, Energy, Mobility, Innovation and Technology (BMK) and for Labour and Economy (BMAW), represented by the Austrian Research Promotion Agency (FFG), and the federal states of Styria, Upper Austria and Tyrol. Markus Böck gratefully acknowledges support from Meta through the grant “Software Assistants for Probabilistic Programming”.

REFERENCES

- [1] Götz Alefeld and Günter Mayer. 2000. Interval analysis: theory and applications. *Journal of computational and applied mathematics* 121, 1-2 (2000), 421–464.
- [2] Gilles Barthe, Joost-Pieter Katoen, and Alexandra Silva. 2020. *Foundations of Probabilistic Programming*. Cambridge University Press.
- [3] Guillaume Baudart, Javier Burroni, Martin Hirzel, Louis Mandel, and Avraham Shinnar. 2021. Compiling Stan to generative probabilistic languages and extension to deep probabilistic programming. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 497–510.
- [4] Ryan Bernstein. 2019. Static analysis for probabilistic programs. *arXiv preprint arXiv:1909.05076* (2019).
- [5] Ryan Bernstein. 2023. *Abstractions for Probabilistic Programming to Support Model Development*. Ph. D. Dissertation. Columbia University.
- [6] Ryan Bernstein, Matthijs Vákár, and Jeannette Wing. 2020. Transforming probabilistic programs for model checking. In *Proceedings of the 2020 ACM-IMS on Foundations of Data Science Conference*. 149–159.

- [7] Mathieu Besançon, Theodore Papamarkou, David Anthoff, Alex Arslan, Simon Byrne, Dahua Lin, and John Pearson. 2021. Distributions.jl: Definition and Modeling of Probability Distributions in the JuliaStats Ecosystem. *Journal of Statistical Software* 98, 16 (2021), 1–30. <https://doi.org/10.18637/jss.v098.i16>
- [8] Eli Bingham, Jonathan P Chen, Martin Jankowiak, Fritz Obermeyer, Neeraj Pradhan, Theofanis Karaletsos, Rohit Singh, Paul Szerlip, Paul Horsfall, and Noah D Goodman. 2019. Pyro: Deep universal probabilistic programming. *The Journal of Machine Learning Research* 20, 1 (2019), 973–978.
- [9] Markus Böck. 2024. LASAPP: Language-Agnostic Static Analysis of Probabilistic Programs. <https://doi.org/10.5281/zenodo.13756900>
- [10] Guillaume Claret, Sriram K Rajamani, Aditya V Nori, Andrew D Gordon, and Johannes Borgström. 2013. Bayesian inference using data flow analysis. In *Proceedings of the 2013 9th joint meeting on foundations of software engineering*. 92–102.
- [11] Keith D Cooper and Linda Torczon. 2011. *Engineering a compiler*. Elsevier.
- [12] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*. 238–252.
- [13] Marco F Cusumano-Towner and Vikash K Mansinghka. 2018. Using probabilistic programs as proposals. *arXiv preprint arXiv:1801.03612* (2018).
- [14] Marco F Cusumano-Towner, Feras A Saad, Alexander K Lew, and Vikash K Mansinghka. 2019. Gen: a general-purpose probabilistic programming system with programmable inference. In *Proceedings of the 40th acm sigplan conference on programming language design and implementation*. 221–236.
- [15] Ron Cytron, Jeanne Ferrante, Barry K Rosen, Mark N Wegman, and F Kenneth Zadeck. 1991. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 13, 4 (1991), 451–490.
- [16] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems: 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29–April 6, 2008. Proceedings 14*. Springer, 337–340.
- [17] Vijay D’silva, Daniel Kroening, and Georg Weissenbacher. 2008. A survey of automated techniques for formal software verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 7 (2008), 1165–1178.
- [18] Robert Dyer, Hoan Anh Nguyen, Hridesh Rajan, and Tien N Nguyen. 2013. Boa: A language and infrastructure for analyzing ultra-large-scale software repositories. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 422–431.
- [19] Robert Dyer, Hoan Anh Nguyen, Hridesh Rajan, and Tien N Nguyen. 2015. Boa: Ultra-large-scale software repository and source-code mining. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 25, 1 (2015), 1–34.
- [20] EvoArt. [n.d.]. HMC issue in the Julia Turing Forum. <https://discourse.julialang.org/t/turing-inexacterror-for-discreteuniform-distribution-with-nuts-sampler/52820>. Accessed: 2023-07-24.
- [21] Hong Ge, Kai Xu, and Zoubin Ghahramani. 2018. Turing: a language for flexible probabilistic inference. In *International conference on artificial intelligence and statistics*. PMLR, 1682–1690.
- [22] Andrew Gelman, John B Carlin, Hal S Stern, and Donald B Rubin. 1995. *Bayesian data analysis*. Chapman and Hall/CRC.
- [23] Andrew D Gordon, Thomas A Henzinger, Aditya V Nori, and Sriram K Rajamani. 2014. Probabilistic programming. In *Future of Software Engineering Proceedings*. 167–181.
- [24] Maria I Gorinova, Andrew D Gordon, and Charles Sutton. 2018. SlicStan: Improving Probabilistic Programming using Information Flow Analysis. In *Workshop on Probabilistic Programming Languages, Semantics, and Systems (PPS)*. <https://pps2018.soic.indiana.edu/files/2017/12/SlicStanPPS.pdf>.
- [25] Maria I Gorinova, Advait Sarkar, Alan F Blackwell, and Don Syme. 2016. A live, multiple-representation probabilistic programming environment for novices. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 2533–2537.
- [26] Anjana Gosain and Ganga Sharma. 2015. Static analysis: A survey of techniques and tools. In *Intelligent Computing and Applications: Proceedings of the International Conference on ICA, 22-24 December 2014*. Springer, 581–591.
- [27] John L Hennessy and David A Patterson. 2011. *Computer architecture: a quantitative approach*. Elsevier.
- [28] Matthew D Hoffman, Andrew Gelman, et al. 2014. The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo. *J. Mach. Learn. Res.* 15, 1 (2014), 1593–1623.
- [29] Gerard J Holzmann. 2017. Cobra: a light-weight tool for static and dynamic program analysis. *Innovations in Systems and Software Engineering* 13, 1 (2017), 35–49.
- [30] Chung-Kil Hur, Aditya V Nori, Sriram K Rajamani, and Selva Samuel. 2014. Slicing probabilistic programs. *ACM SIGPLAN Notices* 49, 6 (2014), 133–144.
- [31] jianlin. [n.d.]. HMC issue in the Pyro Forum. <https://forum.pyro.ai/t/mcmc-discrete-rv-parallelization-is-there-anyway-to-stop-pyro-automatically-vectorizing-tensors/4160>. Accessed: 2023-07-24.
- [32] Alicia A Johnson, Miles Q Ott, and Mine Dogucu. 2022. *Bayes rules: An introduction to applied Bayesian modeling*. Chapman and Hall/CRC.
- [33] James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.
- [34] Daphne Koller and Nir Friedman. 2009. *Probabilistic graphical models: principles and techniques*. MIT press.
- [35] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization*, 2004. CGO 2004. IEEE, 75–86.
- [36] Michael D Lee and Eric-Jan Wagenmakers. 2014. *Bayesian cognitive modeling: A practical course*. Cambridge university press.
- [37] Wonyeol Lee, Hangyeol Yu, Xavier Rival, and Hongseok Yang. 2019. Towards verified stochastic variational inference for probabilistic programs. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–33.
- [38] Alexander K Lew, Marco F Cusumano-Towner, Benjamin Sherman, Michael Carbin, and Vikash K Mansinghka. 2019. Trace types and denotational semantics for sound programmable inference in probabilistic languages. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–32.
- [39] Jianlin Li, Leni Ven, Pengyuan Shi, and Yizhou Zhang. 2023. Type-preserving, dependence-aware guide generation for sound, effective amortized probabilistic inference. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 1454–1482.
- [40] Carol Mak, C-H Luke Ong, Hugo Paquet, and Dominik Wagner. 2021. Densities of almost surely terminating probabilistic programs are differentiable almost everywhere. In *Programming Languages and Systems: 30th European Symposium on Programming, ESOP 2021, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2021, Luxembourg City, Luxembourg, March 27–April 1, 2021. Proceedings 30*. Springer International Publishing, 432–461.
- [41] Carol Mak, Fabian Zaiser, and Luke Ong. 2021. Nonparametric Hamiltonian Monte Carlo. In *International Conference on Machine Learning*. PMLR, 7336–7347.
- [42] Richard McElreath. 2018. *Statistical rethinking: A Bayesian course with examples in R and Stan*. Chapman and Hall/CRC.
- [43] Hadi Mohasel Afshar and Justin Domke. 2015. Reflection, refraction, and hamiltonian monte carlo. *Advances in neural information processing systems* 28 (2015).
- [44] Ramon E Moore. 1966. *Interval analysis*. Vol. 4. Prentice-Hall Englewood Cliffs.
- [45] Chandrakana Nandi, Dan Grossman, Adrian Sampson, Todd Mytkowicz, and Kathryn S McKinley. 2017. Debugging probabilistic programs. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 18–26.
- [46] Akihiko Nishimura, David B Dunson, and Jianfeng Lu. 2020. Discontinuous Hamiltonian Monte Carlo for discrete parameters and discontinuous likelihoods. *Biometrika* 107, 2 (2020), 365–380.
- [47] Aditya Nori, Chung-Kil Hur, Sriram Rajamani, and Selva Samuel. 2014. R2: An efficient MCMC sampler for probabilistic programs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 28.
- [48] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. [arXiv:1912.01703 \[cs.LG\]](https://arxiv.org/abs/1912.01703)
- [49] pymc devs. 2024. pymc-resources. <https://github.com/pymc-devs/pymc-resources/tree/a5f993653e467da11e9fc4ec682e96d59b880102>.
- [50] Caitlin Sadowski, Jeffrey Van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. 2015. Tricorder: Building a program analysis ecosystem. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 598–608.
- [51] John Salvatier, Thomas V Wiecki, and Christopher Fonnesbeck. 2016. Probabilistic programming in Python using PyMC3. *PeerJ Computer Science* 2 (2016), e55.
- [52] Adrian Sampson, Pavel Panchekha, Todd Mytkowicz, Kathryn S McKinley, Dan Grossman, and Luis Ceze. 2014. Expressing and verifying probabilistic assertions. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 112–122.
- [53] StatisticalRethinkingJulia. 2024. SR2TuringPluto. <https://github.com/StatisticalRethinkingJulia/SR2TuringPluto.jl/tree/75072280947a45f030bd45a62710c558d60a2a80>.
- [54] Sam Staton, Hongseok Yang, Frank Wood, Chris Heunen, and Ohad Kammar. 2016. Semantics for probabilistic programming: higher-order functions, continuous distributions, and soft constraints. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science*. 525–534.
- [55] Nazanin Tehrani, Nimar S Arora, Yucen Lily Li, Kinjal Divesh Shah, David Noursi, Michael Tingley, Narjes Torabi, Eric Lippert, Erik Meijer, et al. 2020. Bean machine: A declarative probabilistic programming language for efficient programmable inference. In *International Conference on Probabilistic Graphical Models*. PMLR, 485–496.
- [56] TuringLang. 2024. TuringTutorials. <https://github.com/TuringLang/TuringTutorials/tree/8515a567321ad1f1531974dd14b29c00ea05648>.

- [57] Jan-Willem van de Meent, Brooks Paige, Hongseok Yang, and Frank Wood. 2018. An introduction to probabilistic programming. *arXiv preprint arXiv:1809.10756* (2018).
- [58] Di Wang, Jan Hoffmann, and Thomas Reps. 2021. Sound probabilistic inference via guide types. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 788–803.
- [59] Yuan Zhou, Bradley J Gram-Hansen, Tobias Kohn, Tom Rainforth, Hongseok Yang, and Frank Wood. 2019. LF-PPL: A low-level first order probabilistic programming language for non-differentiable models. In *The 22nd International Conference on Artificial Intelligence and Statistics*. PMLR, 148–157.
- [60] Yuan Zhou, Hongseok Yang, Yee Whye Teh, and Tom Rainforth. 2020. Divide, conquer, and combine: a new inference strategy for probabilistic programs with stochastic support. In *International Conference on Machine Learning*. PMLR, 11534–11545.
- [61] Daniel Zügner, Tobias Kirschstein, Michele Catasta, Jure Leskovec, and Stephan Günnemann. 2021. Language-agnostic representation learning of source code from structure and context. *arXiv preprint arXiv:2103.11318* (2021).